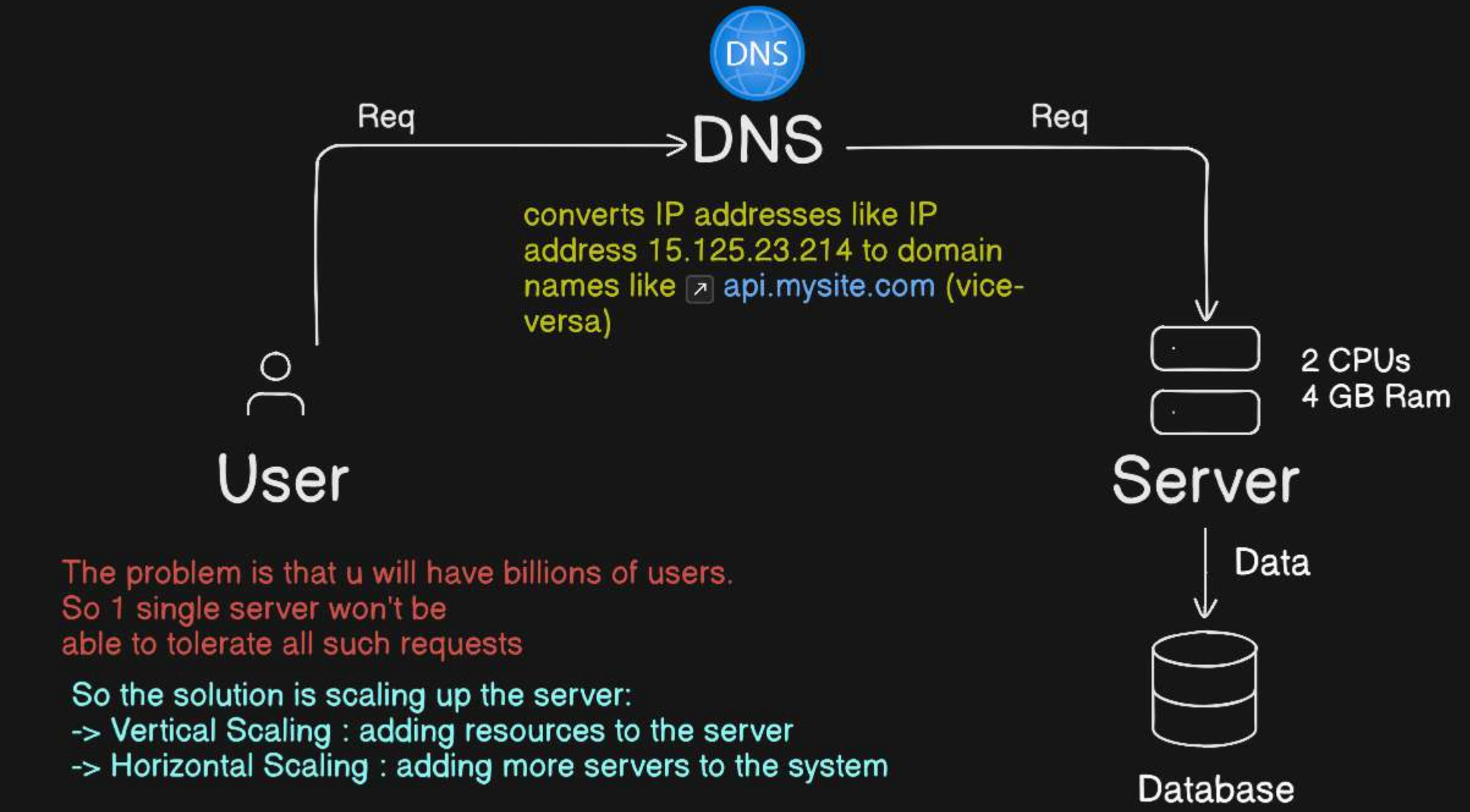
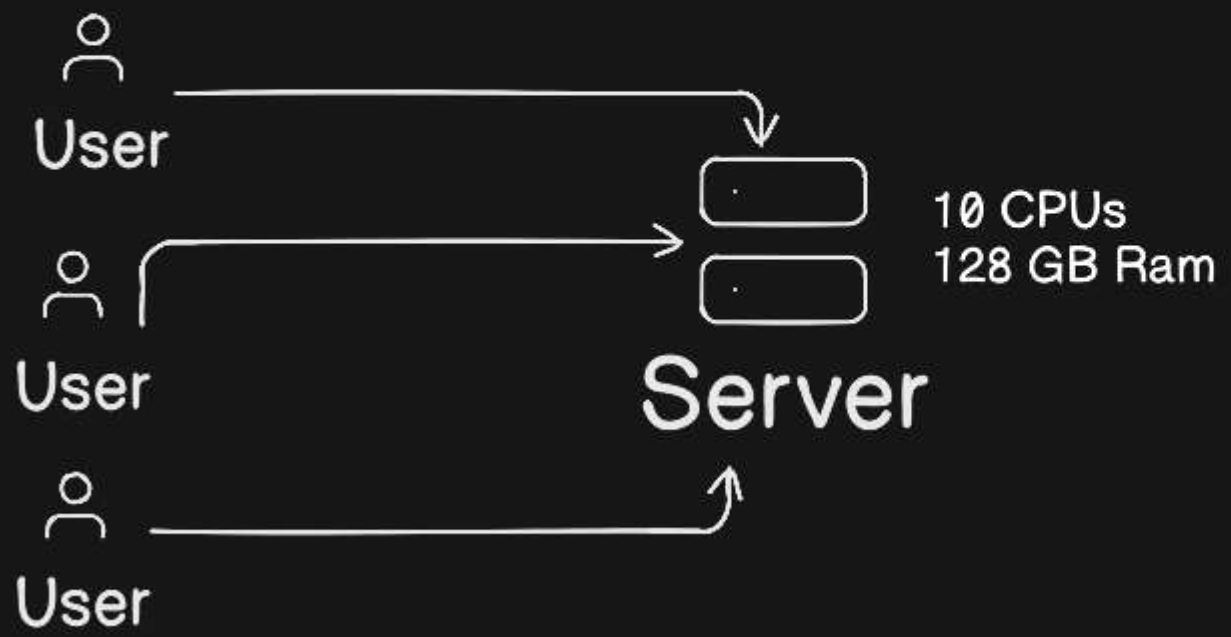
SYSTEM DESIGN BASICS



The problem is that u will have billions of users.
So 1 single server won't be
able to tolerate all such requests

So the solution is scaling up the server:
-> Vertical Scaling : adding resources to the server
-> Horizontal Scaling : adding more servers to the system

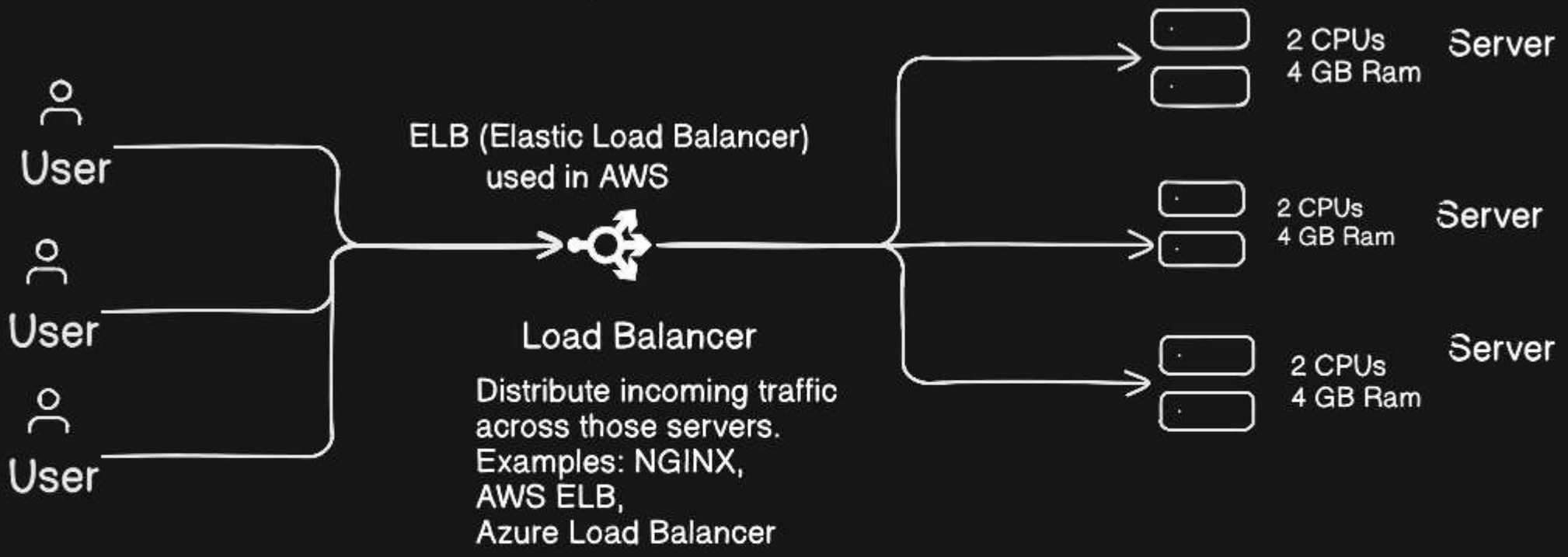
Vertical Scaling



Pros:
Simple to implement
No major architecture change
Easier to manage (single machine)
No distributed system complexity

Cons:
Hardware limit (can't scale infinitely)
Expensive at higher levels
Single point of failure
Downtime required for upgrades

Horizontal Scaling



Pros:
Almost unlimited scalability
High availability (if one fails, others run)
Better fault tolerance
Handles huge traffic

Cons:
More complex architecture
Requires load balancer
Data consistency challenges
Harder debugging

Monolith Architecture vs Microservice Service

Monolith

Single application containing all features:
One codebase
One deployment
Usually one database

Pros:
Simple to build
Easy to deploy
Good for small projects
Lower infrastructure complexity

Cons:
Hard to scale specific parts
Large codebase becomes messy
Slower deployments as it grows

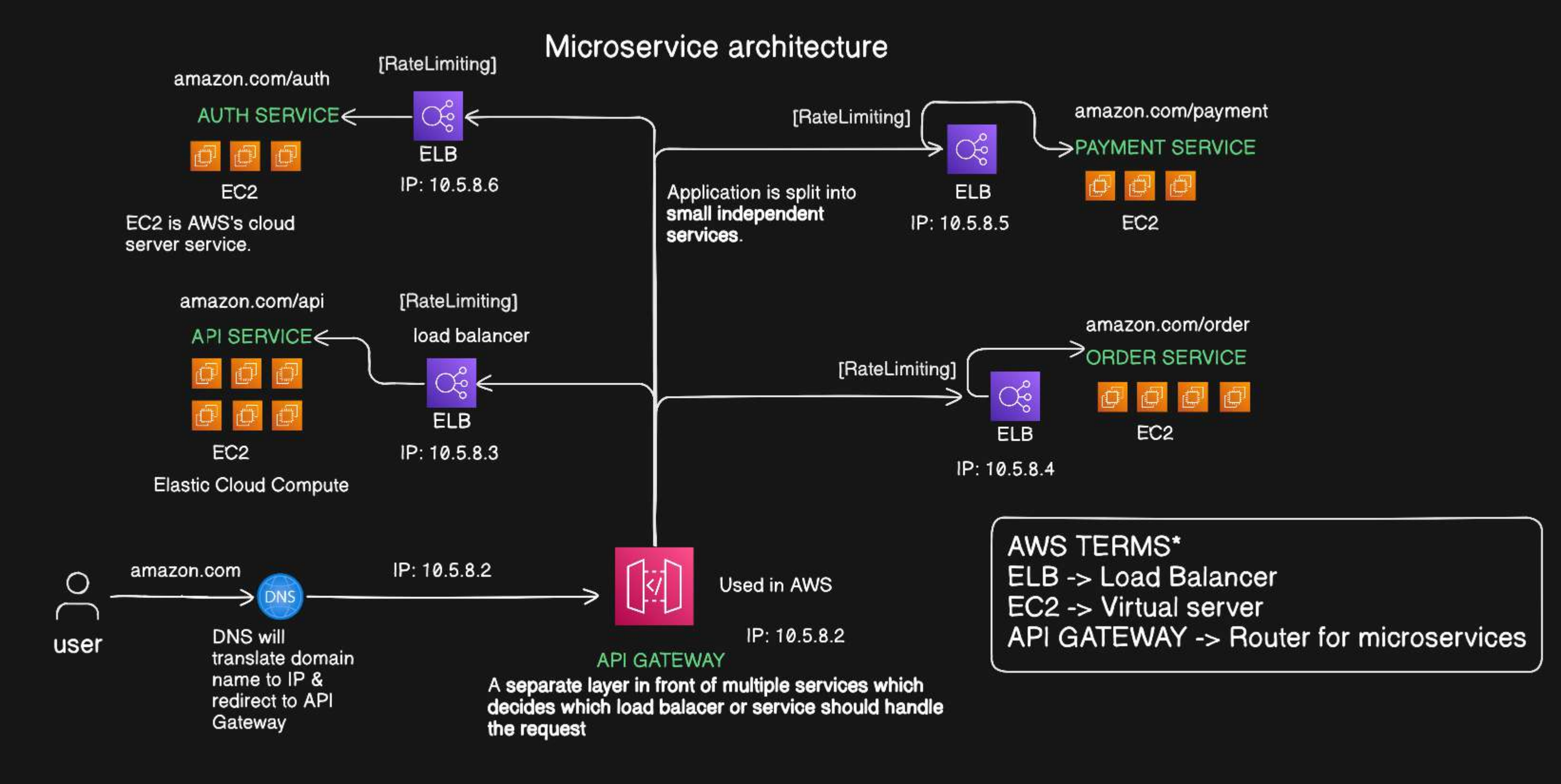
Microservice

Application split into **multiple small independent services**. (request-driven)
Multiple codebases
Independent deployments
Often separate databases

Pros:
Scale services independently
Independent deployment
Better fault isolation

Cons:
More complex
Requires API Gateway, messaging, DevOps
Network & data consistency challenges

Microservice architecture



Important Terminologies

Synchronous Task

The client waits until the task finishes.

Client → Order Service → Save Order → Send Email → Respond

User must wait until:
- Order is saved
- Email is sent

Asynchronous Task

Client does NOT wait for everything to finish.

Client → Order Service → Save Order → Push to Queue → Respond

↓

Email Worker

Order API responds immediately "Order placed successfully"
Email is sent later in background.

A **service**:
- Handles client requests
- Usually exposes APIs
- Responds immediately
- It is **request-driven**.

A **worker**:
- Runs in background
- Processes tasks from a queue
- Not directly exposed to users

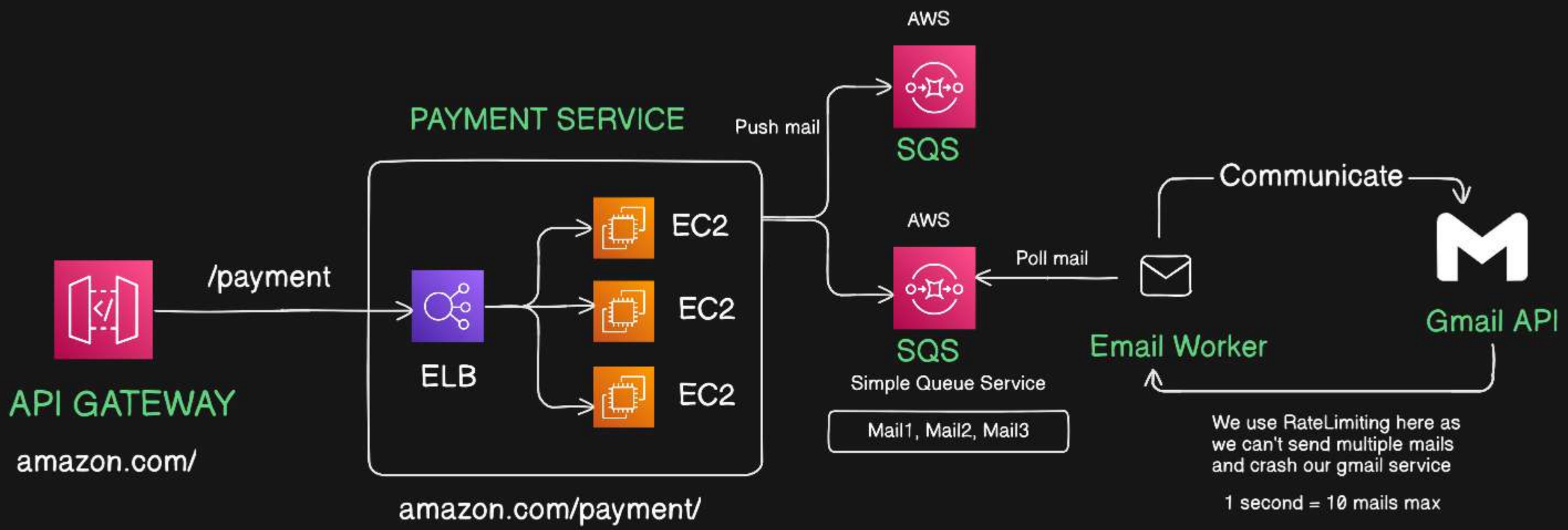
It is **event-driven or task-driven**.

Background job is any task that runs outside the main request cycle.
Example:
- Sending email
- Generating PDF invoice
- Resizing uploaded image

Usually handled by:
- Workers
- Job schedulers

Cron job is scheduled job that runs at specific time intervals.

Runs automatically based on schedule.
Examples:
- Every day at 12 AM → Generate daily sales report
- Every Sunday → Backup database



Here on payment, amazon will send u mail about ur payment.
But the thing is that on payment event, only email workers are listening to this SQS service.

In simpler words, on payment success, u would want to send email, sms, whatsapp to both user and vendor.
So on 1 event, u want to do multiple things. This is possible using Pub-Sub model (event-driven architecture)

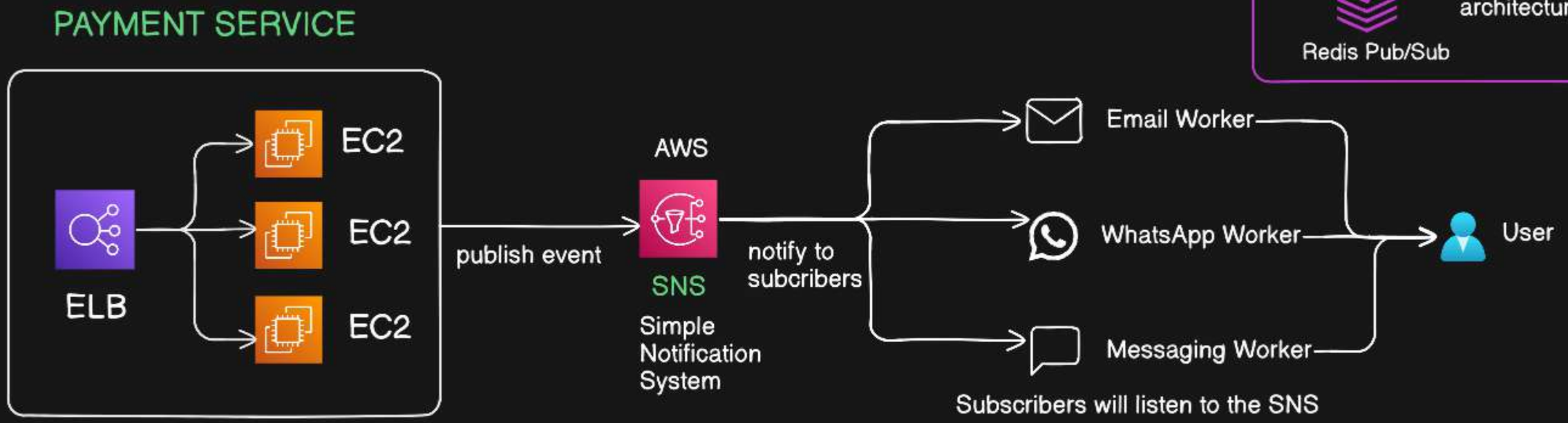
Pub-Sub Model : event-driven architecture

NOTE

Apache Kafka

Redis Pub/Sub

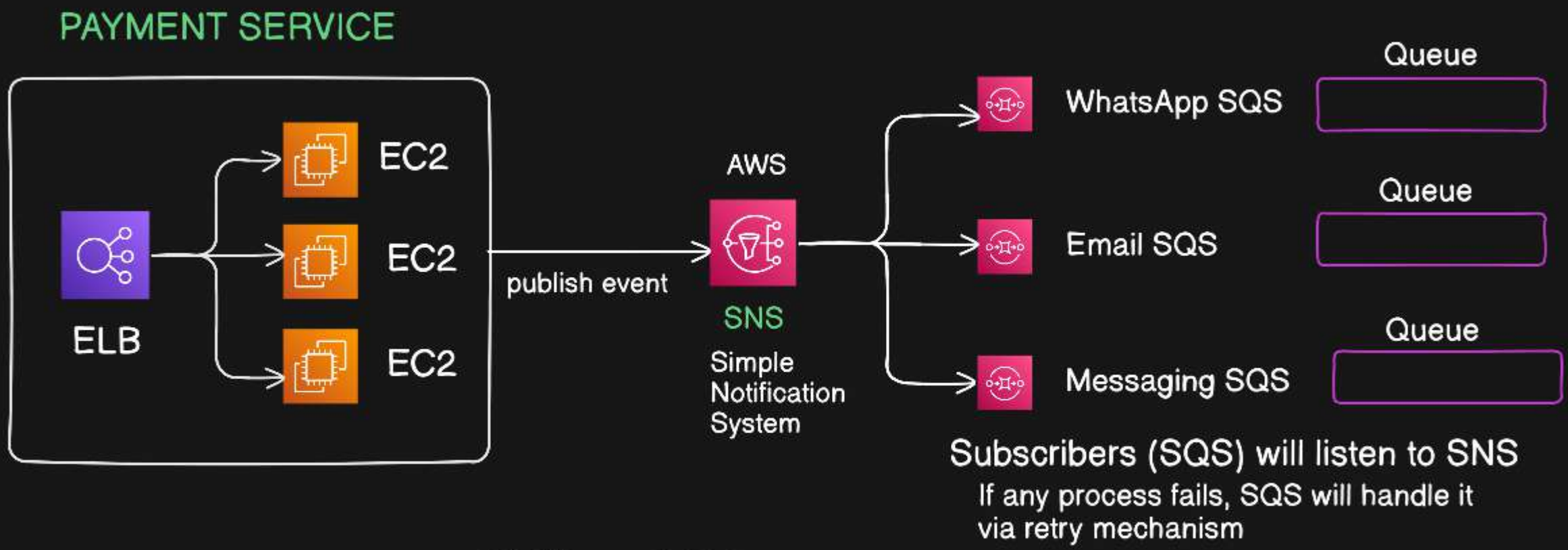
They also follow event driven architecture



Problem in SNS is that u wont get any aknowledgement after the event is published.
It may happen that the whatsapp/email service was down and the user didnt receive any notification.
In SQS (queue system), the user will always get the notification as SQS has a retry mechanism.
Even if the service was down, the mail will be again enqueued in SQS and user will receive the mail after some time.

But again, u simply cant rely on SQS (queue system) for multiple services in 1 event.
To overcome this, we have FAN-OUT architecture

Fan-out Architecture



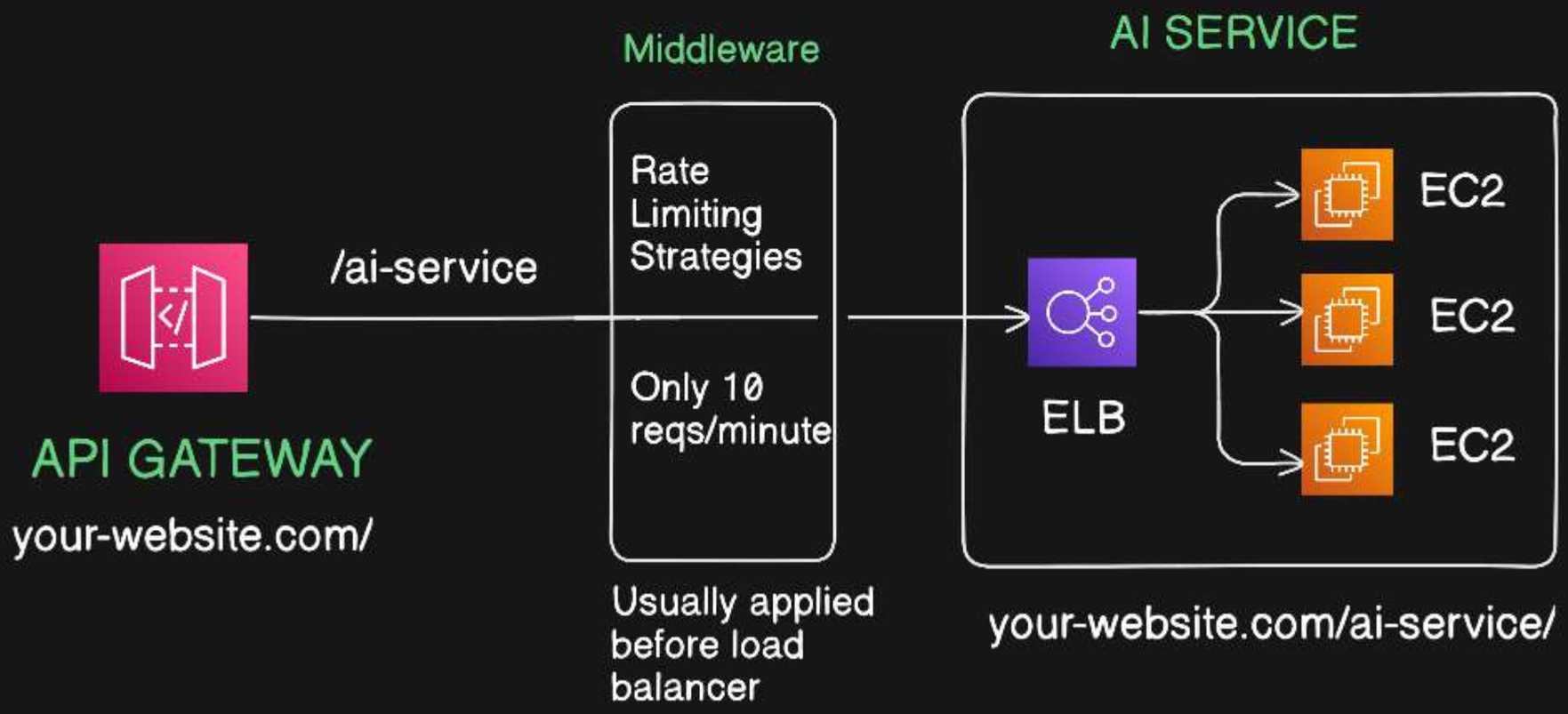
SNS + SQS

SNS for broadcasting and SQS for queueing to manage parallel processing

Rate Limiting

In distributed systems and network management, controlling the flow and rate of the traffic is known as **Rate Limiting**. Rate limiting helps prevent DDoS (Distributed Denial-of-Service) attacks by limiting the number of requests a user or IP address can make within a specific timeframe. DDOS attack -> attempt to disrupt a server, service, or network by overwhelming it with a flood of internet traffic and requests.

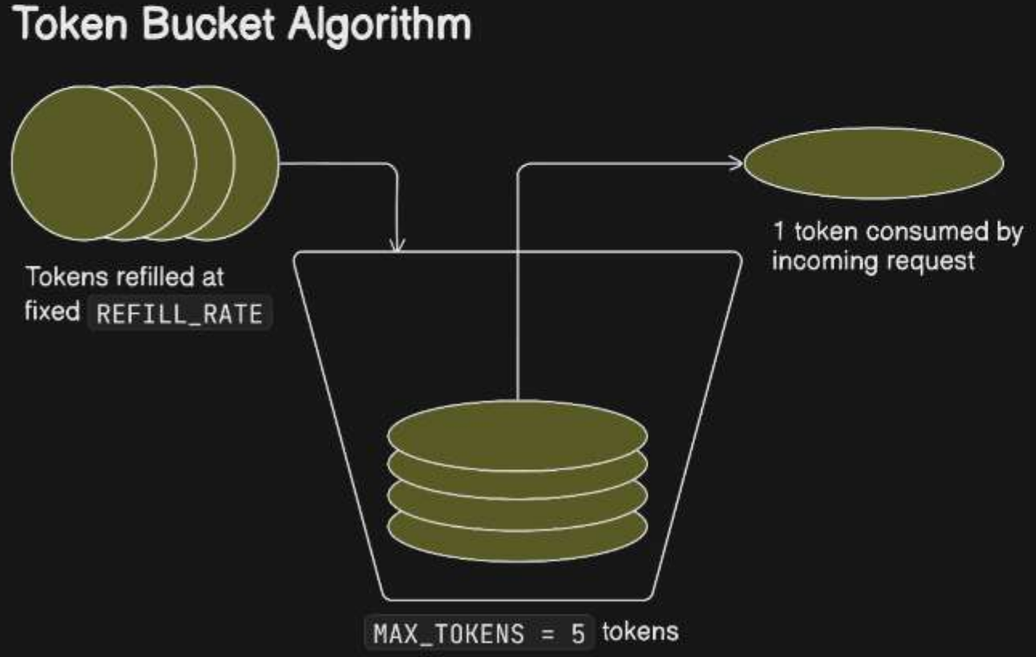
For example, u can only send 10 requests per second to Gmail Service API. This is Rate Limtiing



Rate Limiting Strategies/ALGOs

Token Bucket Algorithm

The Token Bucket algorithm uses a bucket of tokens to limit and regulate the flow of requests. To understand this algorithm, imagine there is a bucket containing a fixed number of tokens. Tokens in this bucket are added at a fixed rate and when a request arrives at the server, a token must be consumed from the bucket. If there are no tokens available, the request can be either rejected or delayed until tokens are available in the bucket.



Pros:
-> Provides more control over the request processing rate with `MAX_CAPACITY` and `REFILL_RATE` as parameters of the token bucket.
-> Allows a short burst of traffic exceeding the `REFILL_RATE` to be processed as long as tokens are available in the bucket.

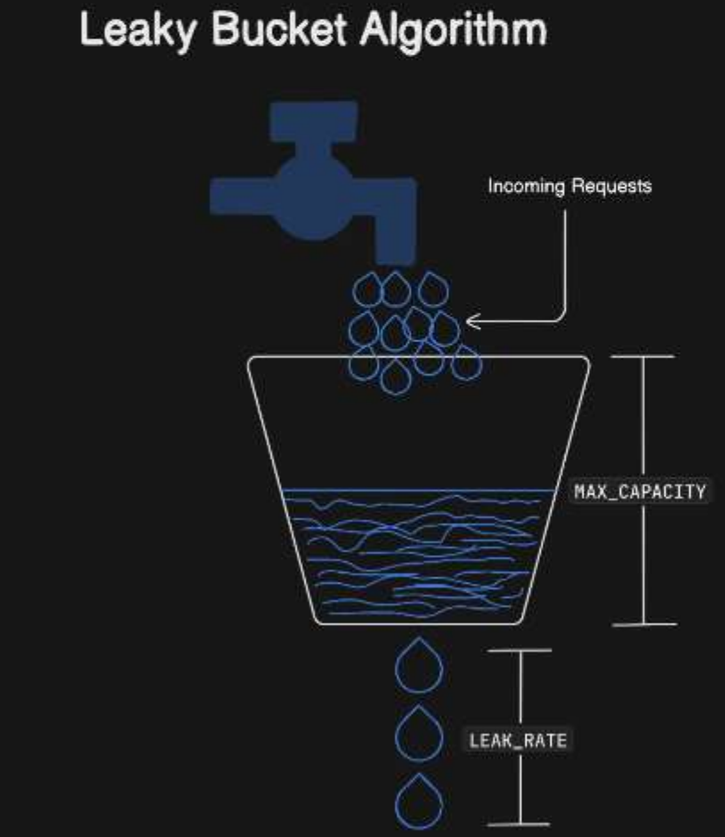
Cons
-> Requires additional overhead for token management.
-> Can be exploited by greedy clients.

Use Token Bucket When:

1. You want more granular control over how fast requests are processed.
2. Your traffic patterns change frequently and you need to be adaptive.
3. You want more control over distributing limited resources among users beyond first-come-first-served.

Leaky Bucket Algorithm

The Leaky Bucket algorithm works on the principle of a physical leaky bucket. Incoming requests are added to the bucket, and the bucket has a fixed capacity. If the bucket overflows, the excess requests are either dropped or delayed. The requests in the bucket are then processed at a fixed rate.
To grasp this idea, think of a bucket filled with water. There's a small hole at the bottom. Water keeps pouring in from the top. If the bucket gets too full, it spills over. The hole at the bottom only lets out 1 drop of water every second, which represents processing speed.



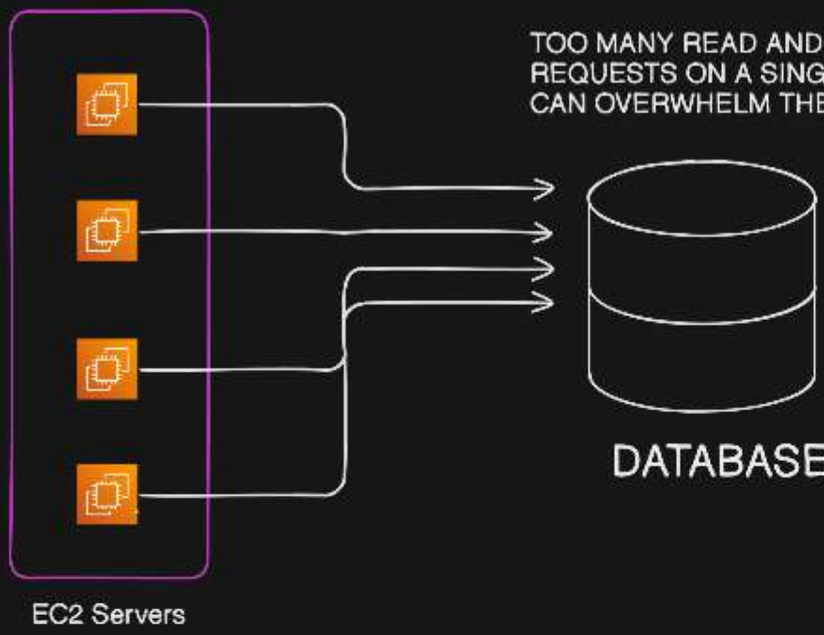
Pros:
-> Constant output which is simply determined by `LEAK_RATE`.
-> Predictable behavior allows for easier maintenance.

Cons
-> Limited flexibility in adjusting to varying traffic patterns (e.g. cannot quickly process small bursts exceeding the `LEAK_RATE`).

Use Leaky Bucket When:

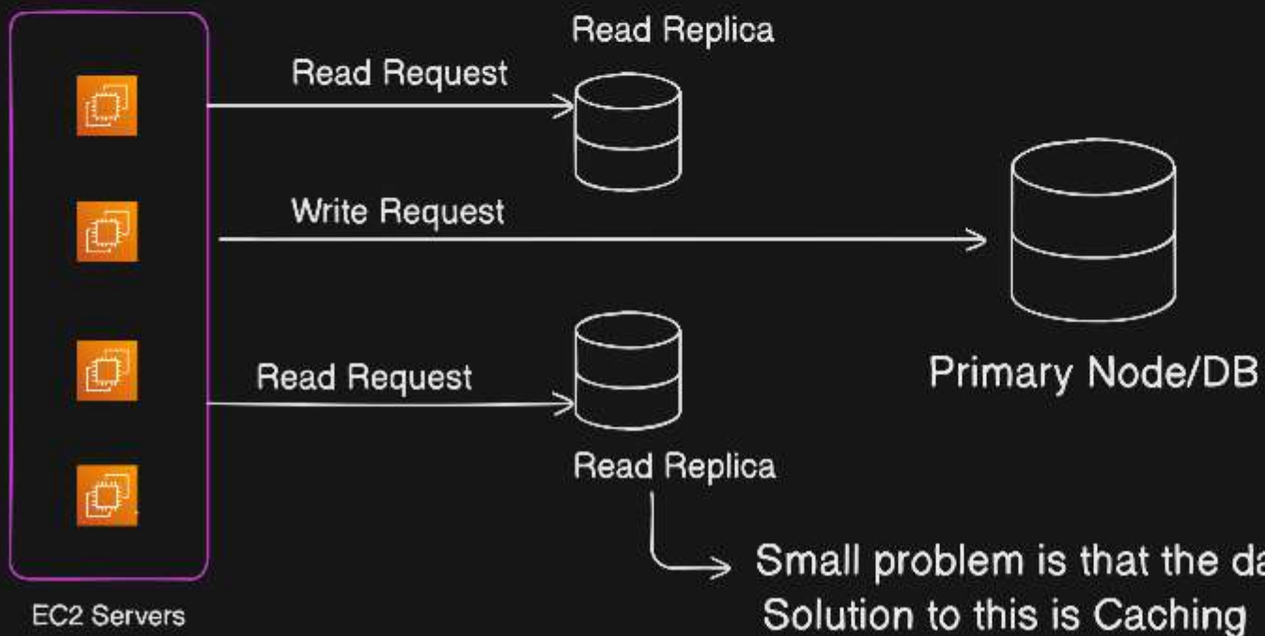
1. You want a straightforward way to manage consistent traffic flow.
2. Keeping your implementation simple and light is a priority.
3. You want to ensure sudden spikes in traffic don't overwhelm your system.

DATABASE REPLICATION



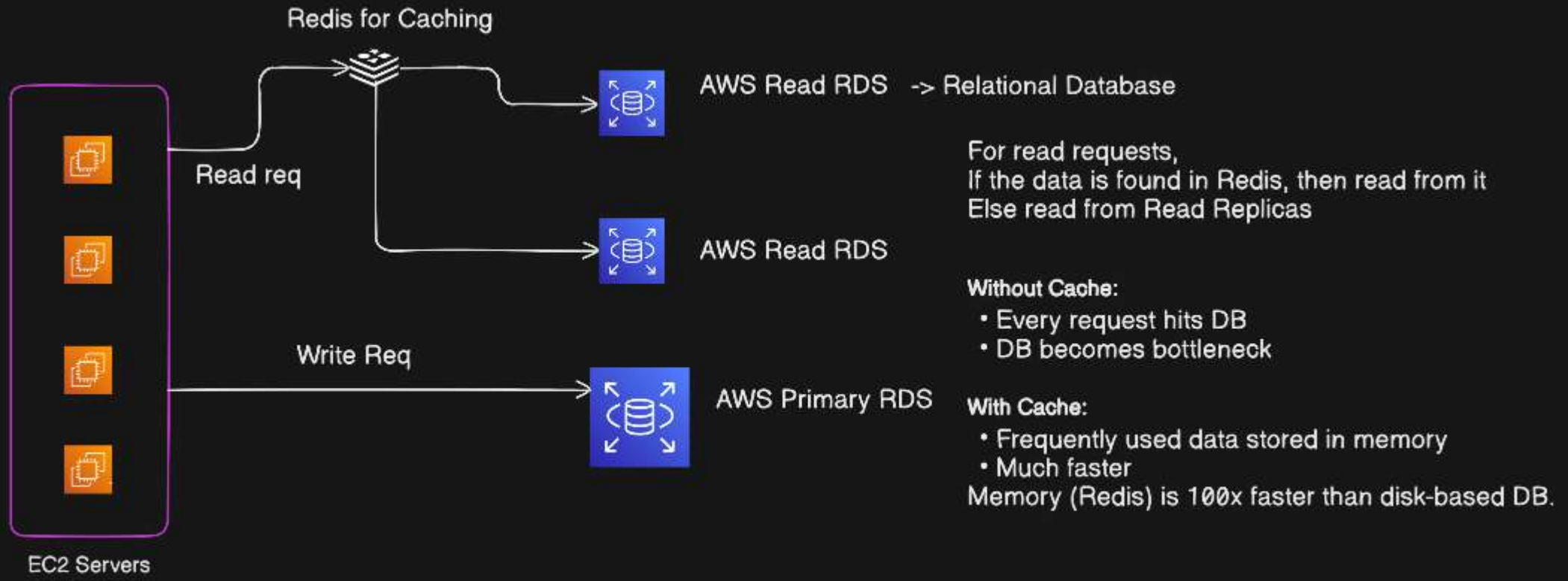
So the best way is to replicate our Database. Meaning, Copying data from one database to other databases. We will have a Master/Slave architecture for Database Replication. Master DB is our Primary DB and other slaves DB are replicas of Master. Changes in primary are copied to replicas.

MASTER/SLAVE ARCHITECTURE



This will help in:
-> High Availability
-> Read Scaling
-> Backup & Disaster Recovery

Small problem is that the data is lil bit delayed here
Solution to this is Caching



For read requests,
If the data is found in Redis, then read from it
Else read from Read Replicas

Without Cache:

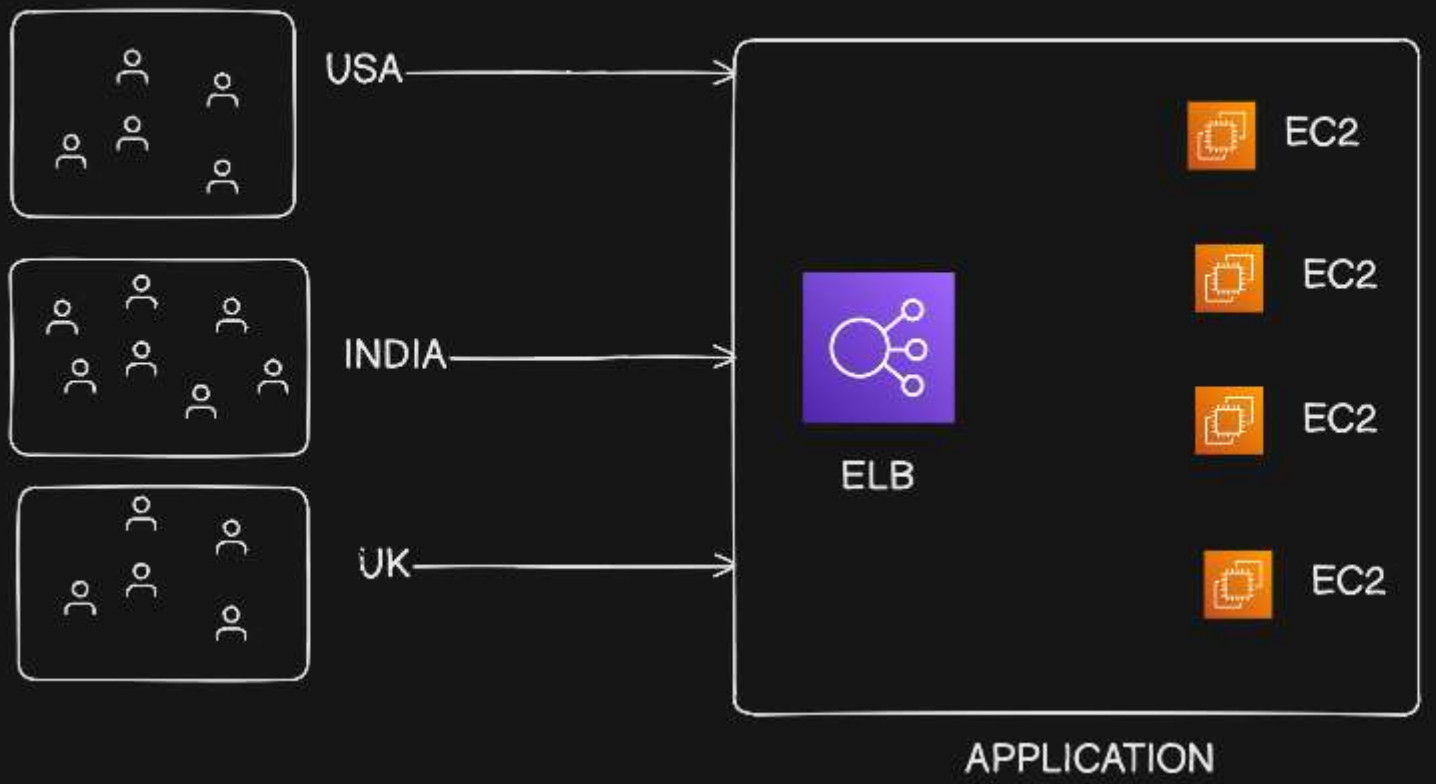
- Every request hits DB
- DB becomes bottleneck

With Cache:

- Frequently used data stored in memory
- Much faster

Memory (Redis) is 100x faster than disk-based DB.

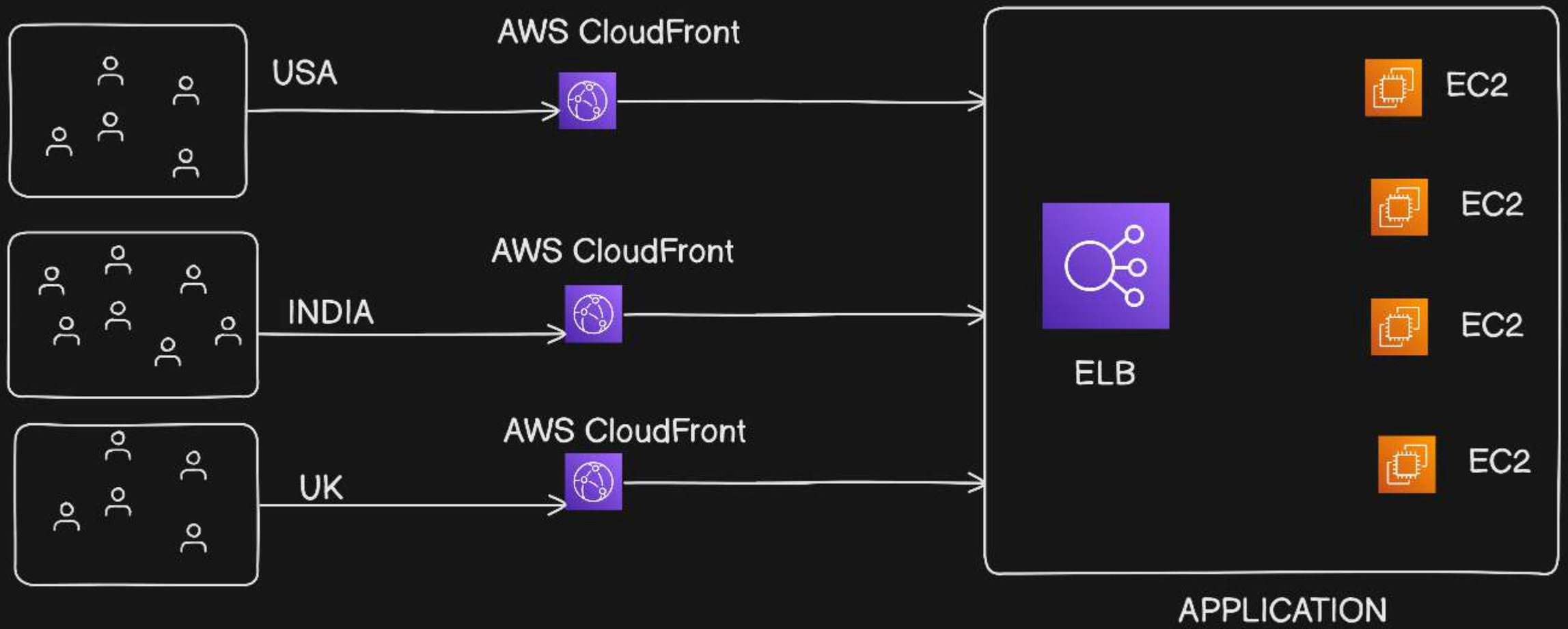
Problem of Millions of Users from Different Regions facing high latency & slow loading time



Too much load on load balancers,since the application has to load static files like Css, images, api responses, videos, pdfs and all, and serve these to users in different locations

Solution : use CDN

CONTENT DELIVERY NETWORK



CloudFront is AWS CDN which is located near users &
caches static content globally and serves users from nearest edge location.
This reduces latency and origin server load , helps in global caching, handles availability.